

مستند معماری کامل ماژول تشخیص ناهنجاری

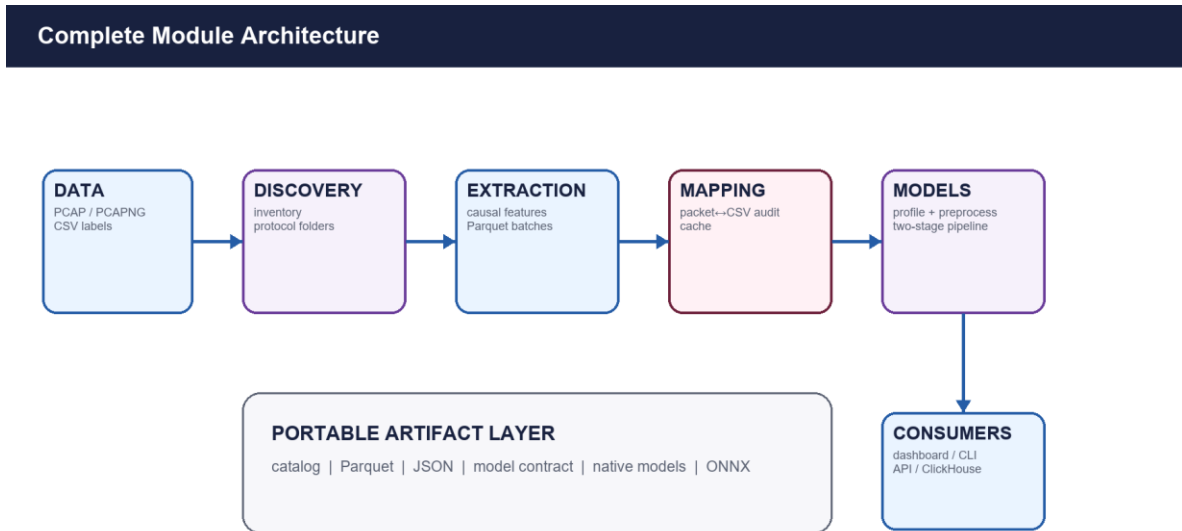
از کشف داده و نگاشت CSV تا قرارداد مدل، داشبورد

نسخه	۱/۰ — بر پایه پیاده‌سازی فعلی
دامنه	سامانه تشخیص ناهنجاری شبکه مبتنی بر PCAP
مخاطب	تیم فنی، ارزیاب معماری و بهره‌بردار سامانه
تاریخ	۱۴۰۵/۰۶/۱۸

این گزارش از روی قراردادها و کدهای فعلی پروژه تولید شده است. نمودارهای داخل سند، مسیرهای پیاده‌سازی شده را نشان می‌دهند و صرفاً تصویر مفهومی نیستند.

۱. نمای کلان و اصول معماری ماژول

ماژول به مرزهای روشن تقسیم شده است: discovery، extraction، mapping، selection، preprocessing، modelling، storage و presentation. هر مرز artifact خود را می‌نویسد؛ مرحله بعدی به داده خام مرحله قبل وابسته نیست و فایل ذخیره‌شده را مصرف می‌کند.



شکل ۱ — معماری کامل و لایه artifact قابل انتقال

نتیجه	اصل
داشبورد، ClickHouse و API مصرف‌کننده‌اند؛ lock-in ایجاد نمی‌شود.	Parquet/JSON source of truth
افزافه کردن protocol یا تحلیل یک protocol بدون قاطعی شدن داده‌ها.	Protocol separation
feature/model یک run در inference دقیقاً قابل تکرار است.	Immutable contracts
سازگاری با WSL، Ubuntu، server بدون GPU.	CPU-first

۲. کشف داده و inventory

ورودی فیزیکی

`datasets/inventory.py` پوشه‌های benign/attack/labels را طبق config مرور می‌کند، فایل‌های PCAP/PCAPNG مستقیم را پیدا می‌کند، protocol را از پوشه تعیین می‌کند و برای هر PCAP حمله candidate CSV می‌سازد. allowlist و override برای کنترل دقیق آزمون یا نام‌گذاری غیرهمسان وجود دارد.

خروجی `input-inventory.json` باید قبل از هر train بررسی شود: نام protocol، capture، مسیر candidate، PCAP‌های CSV و علت حذف احتمالی در آن ثبت می‌شود. این فایل پاسخ می‌دهد آیا داده جدید واقعاً وارد سامانه شده است یا نه.

موجودیت	فیلدهای مهم	مصرف‌کننده
CaptureSource	path, protocol, split, capture name	catalog و extractor
AttackPair	attack PCAP, candidate CSV, protocol	dataset mapper
allowlist	نام capture مجاز	smoke/reprocess هدفمند
override	pair دستی PCAP/CSV	دیتاست با نام‌های ناهمسان

۳. استخراج feature از PCAP

مسئولیت extractor

`features/extractor.py` با Scapy پکت را می‌خواند، IP/port/payload/زمان را نرمال می‌کند، protocol واقعی را تشخیص می‌دهد و یک row پایه ایجاد می‌کند. سپس burst، timing، direction، state causal flow و host behavior را به row اضافه می‌کند. هیچ feature به آینده همان capture نگاه نمی‌کند.

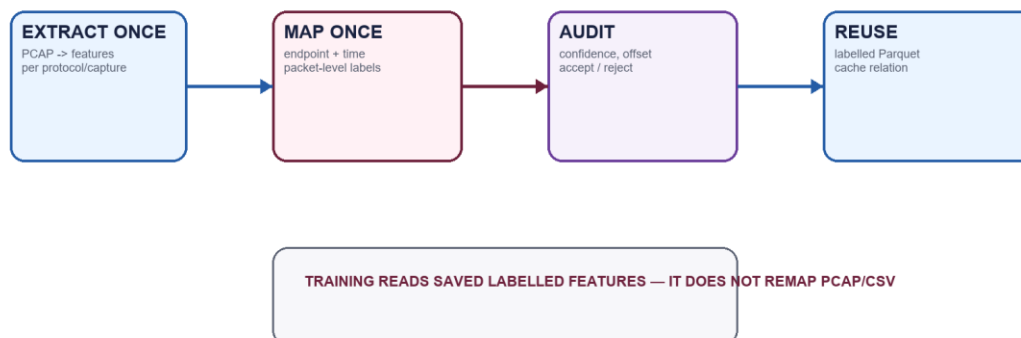
برای capture بزرگ، extractor به صورت streaming کار می‌کند: Parquet row group نوشته می‌شود، state flow/host سقف دارد، memory کنترل می‌شود و progress انگلیسی در console و JSON ثبت می‌شود. برای smoke sample یکنواخت و قطعی گرفته می‌شود.

نمونه output	لایه
packet_length, payload_size, entropy, TCP flags	Packet
flow_duration, total packets/bytes, ratio, mean/std	Flow
inter-arrival, jitter, packet rate, hour/weekday cycles	Timing
source rate, destination count, entropy, burstiness	Host behaviour
DNS/HTTP/Modbus/Sycomm/SSH fields	Protocol parser

۴. معماری mapping و cache برچسب

نگاشت بعد از extraction اجرا می‌شود و همان records ذخیره‌شده را می‌خواند؛ PCAP برای train مجدداً parse نمی‌شود. mapperهای متفاوت CSV را normalize می‌کند، endpoint/time سازگار را پیدا می‌کند، offset زمانی محتمل را audit می‌کند و label را در زمان هر packet attach می‌کند.

Data, Mapping, and Cache Lifecycle



شکل ۲ — lifecycle استخراج، audit نگاشت، cache و استفاده بعدی

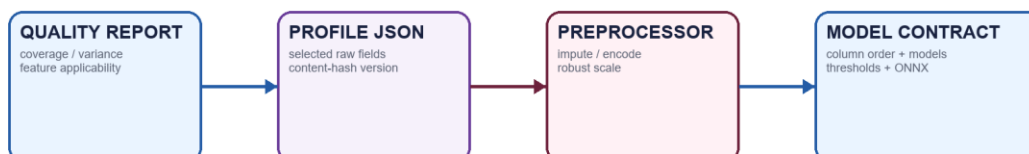
cache رابطه packet↔CSV را با signature منبع و policy نگه می‌دارد. cache mismatch باعث rebuild همان capture می‌شود، نه این‌که label با ترتیب row فرض شود. mapping پذیرفته‌شده و label معتبر دو مفهوم جدا هستند: یک capture پذیرفته‌شده می‌تواند packetهای unknown داشته باشد.

قانون	اثر
timestamp + endpoint evidence	کاهش اتصال اشتباه برچسب CSV به PCAP.
mapping audit	امکان رد رابطه کم‌اعتماد قبل از train.
unknown حفظ می‌شود	عدم تبدیل packet بدون evidence به benign یا attack.
cache reuse	اجرای سریع‌تر و نتیجه قابل تکرار در runهای بعدی.

۵. انتخاب feature و قرارداد preprocessing

feature catalog را با protocol، توضیح و cost ثبت می‌کند. `select analyze` وضعیت evidence usable/constant/near_constant/low_coverage/not_observed/not_implemented را محاسبه می‌کند. کاربر پس از دیدن این profile یک نسخه‌دار ایجاد می‌کند.

Feature Profile and Inference Contract



Inference uses the saved contract exactly; a dashboard selection cannot change a deployed feature set.

شکل ۳ — profile نسخه‌دار تا model contract inference

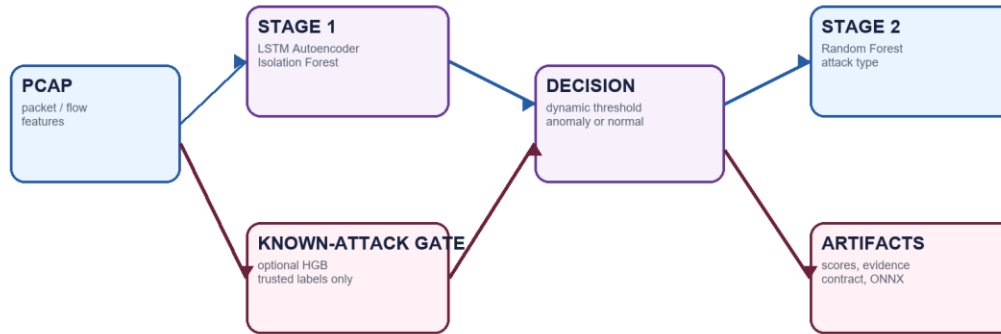
`preprocessing/pipeline.py` فقط روی training rows fit می‌شود. impute، numeric، category، robust-scale، محدود و one-hot encode می‌شوند. manifest تمام featureهای انتخابی، حذف‌شده، vocabulary، مدل columnهای transform شده و ترتیب آن‌ها را ثبت می‌کند.

artifact	چرا مهم است؟
profile JSON	انتخاب feature کاربر و نسخه آن.
quality report	دلیل قابل دفاع برای نگه‌داشتن یا حذف feature.
pipeline.joblib	خود transformation فیت‌شده.
manifest JSON	schema و column order مورد انتظار مدل.
model-contract.json	اتصال profile/preprocessor به مدل deploy شده.

۶. pipeline مدل، ارزیابی و خروجی تحلیلی

پس از فریز feature contract، مدل دومرحله‌ای اجرا می‌شود. Stage ۱ normal-only LSTM-AE و Isolation Forest را fit می‌کند، threshold را روی normal calibration می‌سازد و در صورت label captureهای کافی گیت supervised سبک را اضافه می‌کند. Stage ۲ نوع حمله را فقط برای anomaly candidateها پیش‌بینی می‌کند.

Two-Stage Detection Architecture



Novel traffic remains covered by Stage 1 even when Stage 2 is unavailable.

شکل ۴ — pipeline مدل در مرز ماژول

هر خروجی تحلیلی بیرون از dashboard نیز وجود دارد: score پکت، readiness، history LSTM، evidence feature، prediction و metric RF. threshold sensitivity، end-to-end، latency، resource usage، dashboard و همان Parquet/JSONها را visual می‌کند.

خروجی	کاربرد سیستم دیگر
scores.parquet	سامانه SIEM، alert، یا پرونده incident.
feature-evidence.parquet	توضیح انسانی/گزارش SOC.
end-to-end-predictions.parquet	نتیجه normal/anomaly/type برای API یا DB.
model-contract.json	بارگذاری امن مدل در worker inference.
training-summary.json	نمایش KPI و رصد release مدل.

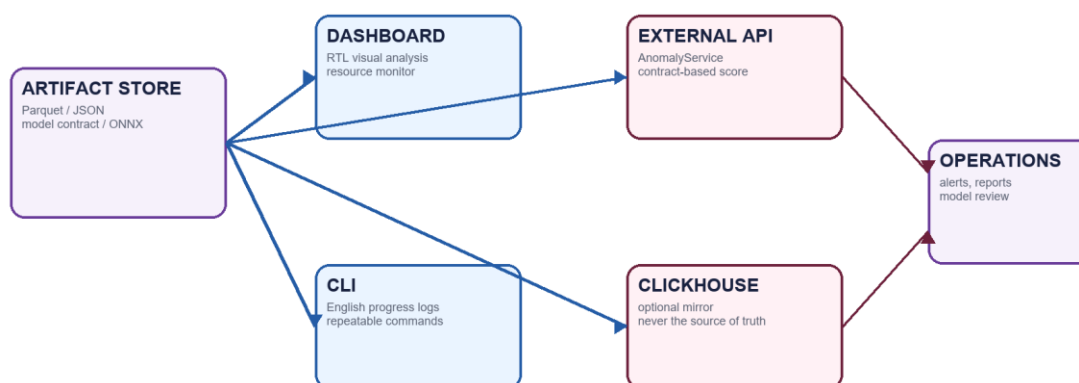
۷. ذخیره‌سازی artifact و ClickHouse

منبع حقیقت

`storage/artifacts.py` ابتدا artifact را به صورت فایل محلی می نویسد و در `catalog/artifacts.parquet` ثبت می کند. این انتخاب باعث می شود تغییر یا قطع ClickHouse، مسیر extraction/train/inference را متوقف نکند. schema hash و description به مصرف کننده برای اعتبارسنجی کمک می کنند.

ClickHouseMirror فقط وقتی در config فعال شود ساخته می شود. artifact های data frame را به جدول های سازگار mirror می کند، اما هیچ pipeline برای خواندن حیاتی به آن وابسته نیست. credential با environment variable خوانده می شود، نه از source code.

Artifact Consumers and Operational Monitoring



شکل ۵ — مصرف artifact ها توسط CLI، API، dashboard و ClickHouse

`dashboard/workbench.py` یک workbench فارسی RTL است. تحلیل کلی و per-protocol، نمودارهای mapping/feature/model، جدول‌های evidence، نمودار pipeline، download artifact، live training monitor و resource monitor در این لایه است. dashboard با انتخاب run و protocol فقط فایل‌های local را می‌خواند.

`cli.py` همان قابلیت‌ها را بدون dashboard عرضه می‌کند و log‌های طولانی را انگلیسی نگه می‌دارد تا برای terminal/server قابل پردازش باشند. `service.py` façade برنامه‌نویسی است که برای product دیگر، API داخلی یا batch worker مناسب است.

رابط	ورودی	خروجی
CLI	path/config/options	event log و artifact path، summary
Dashboard	artifact root و انتخاب اپراتور	نمودار/جدول/دانلود بدون تغییر source data
AnomalyService	PCAP/profile/model artifact	path‌های صریح model/scores/metrics
ClickHouse	mirror اختیاری frameها	table برای query سازمانی، نه truth اصلی.

استقرار Linux/WSL

پروژه با Python ۳.۱۱/۳.۱۲ و `uv` قابل اجرا است. برای server بدون اینترنت، uv cache/wheelhouse، Python runtime، `uv` binary و کل پوشه پروژه از Windows/WSL منتقل می‌شود، سپس `uv sync --offline --frozen` اجرا می‌شود. runtime پیش‌فرض CPU است.

سیاست منابع

`capture.streaming`، `memory\_limit\_gb`، `\*\_runtime.cpu\_workers` و `models.max\_source\_rows` مرزهای اجرای سنگین را کنترل می‌کنند. extraction تمام داده می‌تواند disk-backed باشد، درحالی‌که model fit با sample نماینده و محدود انجام می‌شود. progress دوام‌دار مانع انتظار بدون اطلاع کاربر می‌شود.

تست و تغییر امن

- تست preprocess، selection، cache، mapping، protocol parser، مدل‌ها و dashboard در `/tests` جدا شده‌اند.
- افزودن protocol مستلزم config، feature catalog، parser، تست و راهنمای جدید است.
- تغییر schema artifact باید هم‌زمان writer، catalog، dashboard reader و integration guide را به‌روز کند.
- قبل از release مدل، normal holdout FPR، mapping audit و capture-held-out attack metrics باید بازبینی شوند.